

SwiftUI + SwiftData App Entry Notes

App Entry Point

```
@main
struct todo_appApp: App
```

- `@main` = starting point of app
- Similar to Flutter:

```
runApp(MyApp())
```

- `App` protocol defines the app lifecycle

ModelContainer

```
var sharedModelContainer: ModelContainer
```

- Main SwiftData storage manager
- Handles:
 - Save
 - Fetch
 - Persistence
 - Model registration

Similar to:

- SQLite database manager
- Core Data persistent container

Schema

```
let schema = Schema([
    Item.self,
])
```

- Defines app data models
- Similar to database structure

SwiftData	Database
Model	Table
Schema	Database structure

`Item.self` registers the `Item` model.

ModelConfiguration

```
let modelConfiguration = ModelConfiguration(
    schema: schema,
    isStoredInMemoryOnly: false
)
```

Controls how data is stored.

isStoredInMemoryOnly

false

- Data stored permanently on disk
- Persists after app restart

true

- Temporary in-memory storage
- Data removed when app closes
- Useful for testing/previews

Creating ModelContainer

```
return try ModelContainer(
    for: schema,
    configurations: [modelConfiguration]
)
```

SwiftData: 1. Reads schema 2. Creates storage 3. Registers models 4. Prepares persistence system

Error Handling

```
catch {  
  fatalError("Could not create ModelContainer: \(error)")  
}
```

- Crashes app if storage setup fails
 - Useful during development
-

WindowGroup

```
WindowGroup {  
  ContentView()  
}
```

- Creates app's main window/scene
- `ContentView()` becomes root screen

Similar to Flutter:

```
MaterialApp(  
  home: ContentView(),  
)
```

Injecting SwiftData

```
.modelContainer(sharedModelContainer)
```

Injects SwiftData container into all child views.

Views can access:

- ModelContext
- Queries
- Insert/update/delete operations

Similar to:

- Provider
- Dependency Injection
- Riverpod global provider

App Startup Flow

```
App Launch
↓
@main executes
↓
Schema created
↓
ModelConfiguration created
↓
ModelContainer initialized
↓
WindowGroup created
↓
ContentView loaded
```

Flutter vs SwiftUI Mapping

SwiftUI	Flutter
@main	main()
App	runApp()
WindowGroup	MaterialApp
ContentView	Root Widget
ModelContainer	Database setup
.modelContainer()	Provider injection

Quick Summary

@main starts the app, WindowGroup creates the UI, ContentView is the root screen, and ModelContainer manages SwiftData persistence.